

Universidad CENFOTEC

Principios de Programación 1 (SOFT-01) | Tarea 2

Problemática: Cajero Automático

Docente: Veronica Mora Lezcano

Estudiante: Eddie Steven Mora Mesén

Fecha de entrega: 25 de octubre 2026

Periodo: 2026-C3

Repositorio:

https://github.com/emorame00/cajero_automatico

Rama del repositorio: main

Problema seleccionado

Problema 1. Cajero automático.

La solución permite realizar retiros sobre un saldo inicial de ₡250 000. Cada retiro debe estar entre ₡5 000 y ₡100 000 y no puede superar el saldo disponible. Los intentos rechazados no descuentan dinero ni aumentan el contador de retiros.

Se pregunta al usuario si desea comenzar y, cuando todavía hay fondos suficientes, si desea continuar. El proceso finaliza cuando responde n o cuando el saldo queda por debajo de ₡5 000. Se informa el resultado de cada intento, el saldo actualizado y un resumen final con el saldo, la cantidad de retiros exitosos y el total retirado.

Análisis del algoritmo

Entradas: monto solicitado y confirmaciones s/n. Procesamiento: comparar el monto con los límites y el saldo; descontar retiros válidos; incrementar el contador y acumular el dinero retirado. Salidas: mensajes de aceptación o rechazo, saldo actualizado y resumen final.

El ciclo while se utiliza porque la cantidad de intentos no se conoce de antemano. Su condición es continuar == "s" and saldo >= RETIRO_MINIMO. El contador retiros_exitosos aumenta en uno y el acumulador total_retirado suma el monto únicamente cuando el retiro es válido.

Diagrama general

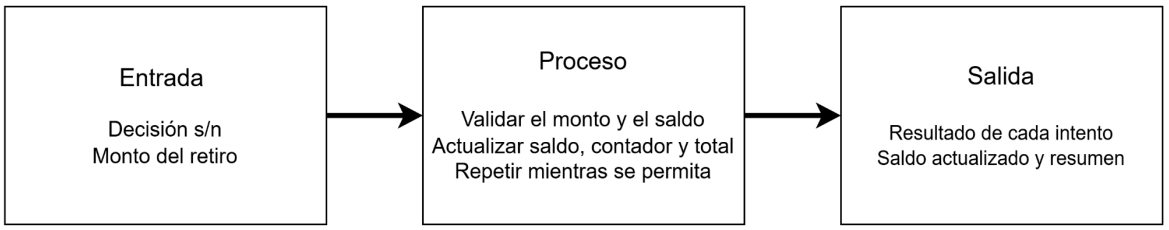
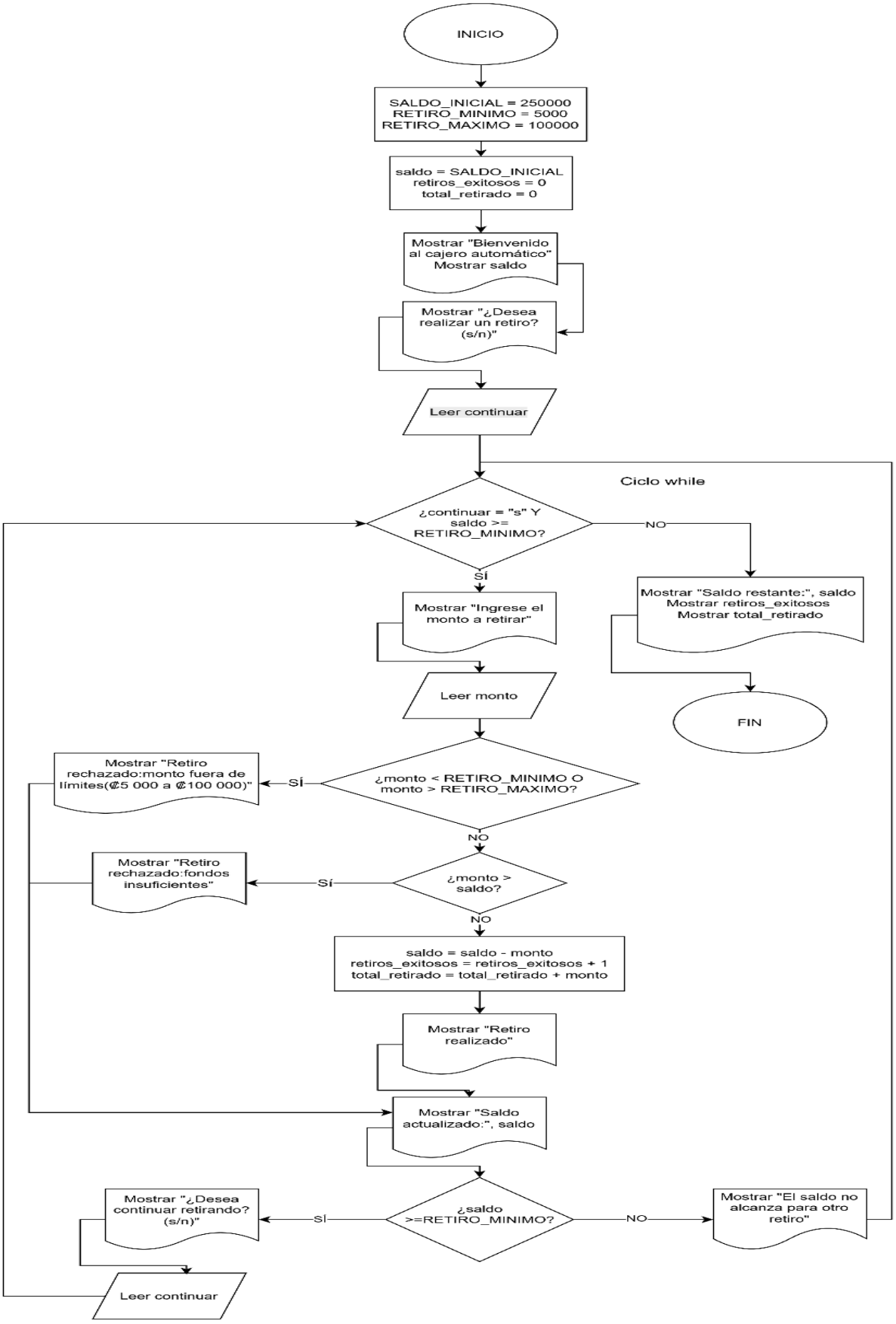


Diagrama de flujo



Código fuente en python

CÓDIGO

```
# Constantes: reglas del cajero.
SALDO_INICIAL = 250000
RETIRO_MINIMO = 5000
RETIRO_MAXIMO = 100000

# Variables: saldo, contador y acumulador.
saldo = SALDO_INICIAL
retiros_exitosos = 0
total_retirado = 0

print("Bienvenido al cajero automático")
print("Saldo inicial: ₡", saldo)
continuar = input("¿Desea realizar un retiro? (s/n): ").strip().lower()

# Repetir mientras el usuario quiera y el saldo alcance para retirar.
while continuar == "s" and saldo >= RETIRO_MINIMO:
    monto = float(input("Ingrese el monto a retirar: "))

    if monto < RETIRO_MINIMO or monto > RETIRO_MAXIMO:
        print("Retiro rechazado: monto fuera de límites.")
        print("Retiro mínimo: ₡", RETIRO_MINIMO)
        print("Retiro máximo: ₡", RETIRO_MAXIMO)
    elif monto > saldo:
        print("Retiro rechazado: fondos insuficientes.")
    else:
        saldo = saldo - monto
        retiros_exitosos = retiros_exitosos + 1
        total_retirado = total_retirado + monto
        print("Retiro realizado.")

    print("Saldo actualizado: ₡", saldo)

    if saldo >= RETIRO_MINIMO:
        continuar = input("¿Desea continuar retirando? (s/n): ")
        continuar = continuar.strip().lower()
    else:
        print("El saldo no alcanza para otro retiro.")

# Resumen: se muestra después de salir del ciclo.
print("\nResumen de operaciones")
print("Saldo restante: ₡", saldo)
print("Cantidad de retiros exitosos:", retiros_exitosos)
print("Total de dinero retirado: ₡", total_retirado)
```

Datos de entrada y salida

Entrada	Variable	Tipo	Uso
Confirmación inicial y posterior	continuar	str	Recibe s/n; se eliminan espacios externos y se convierte a minúsculas.
Monto solicitado	monto	float	Se compara con los límites y el saldo. Se ingresa un número sin símbolos ni separadores.

Salida	Dato o variable	Momento
Bienvenida y saldo inicial	saldo	Antes del ciclo.
Aceptación o rechazo	Mensaje según las condiciones	En cada intento.
Saldo actualizado	saldo	Después de cada intento.
Aviso de saldo insuficiente para otro retiro	Mensaje	Cuando saldo < RETIRO_MINIMO.
Resumen final	saldo, retiros_exitosos y total_retirado	Al salir del ciclo.

Constantes y variables internas

Nombre	Valor inicial	Función
SALDO_INICIAL	250000	Saldo inicial fijo.
RETIRO_MINIMO	5000	Límite inferior de cada retiro.
RETIRO_MAXIMO	100000	Límite superior de cada retiro.
saldo	250000	Dinero disponible; disminuye con retiros válidos.
retiros_exitosos	0	Contador; suma 1 por retiro válido.
total_retirado	0	Acumulador; suma cada monto retirado.

Prueba de escritorio del caso normal

Entradas: s → 20000 → s → 30000 → n. Esta traza simula manualmente dos retiros válidos. En las tablas, contador corresponde a retiros_exitosos y total corresponde a total_retirado.

Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
0 (inicial)	continuar = "s" saldo = 250000	Todavía no se evalúa.	—	saldo = 250000 contador = 0 total = 0 Se lee continuar = "s".	contador = 0 total = 0	Bienvenida, saldo inicial y pregunta para comenzar.
1	continuar = "s" saldo = 250000	"s" == "s" Y 250000 >= 5000: verdadero.	Sí	monto = 20000 20000 < 5000 O 20000 > 100000: falso. 20000 > 250000: falso. saldo: 250000 - 20000 = 230000 contador: 0 + 1 = 1 total: 0 + 20000 = 20000 230000 >= 5000: verdadero. Se lee continuar = "s" (antes: "s").	contador = 1 total = 20000	Retiro realizado. Saldo: ₡230 000. Pregunta si desea continuar.
2	continuar = "s" saldo = 230000	"s" == "s" Y 230000 >= 5000: verdadero.	Sí	monto: 20000 → 30000 30000 < 5000 O 30000 > 100000: falso. 30000 > 230000: falso. saldo: 230000 - 30000 = 200000 contador: 1 + 1 = 2 total: 20000 + 30000 = 50000 200000 >= 5000: verdadero. continuar: "s" → "n".	contador = 2 total = 50000	Retiro realizado. Saldo: ₡200 000. Pregunta si desea continuar.
Final	continuar = "n" saldo = 200000	"n" == "s": falso. La condición completa es falsa.	No	No se ejecuta el cuerpo. Valores sin cambios.	contador = 2 total = 50000	Saldo restante: ₡200 000. Retiros: 2. Total retirado: ₡50 000.

Causa de terminación: continuar cambia de "s" a "n". Python deja de evaluar and al encontrar falsa su primera condición.

Traza del caso con cero iteraciones

Entrada: n. La pregunta inicial permite salir sin solicitar ningún monto.

Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
0 (inicial)	continuar = "n" saldo = 250000	Todavía no se evalúa.	—	saldo = 250000; contador = 0; total = 0. Se lee continuar = "n".	contador = 0 total = 0	Bienvenida, saldo inicial y pregunta para comenzar.
Final	continuar = "n" saldo = 250000	"n" == "s": falso.	No	No se solicita monto ni se actualizan los valores.	contador = 0 total = 0	Saldo: ₡250 000. Retiros: 0. Total: ₡0.

Causa de terminación: la respuesta inicial es "n". El cuerpo del while se ejecuta cero veces.

Traza del caso con una iteración

Entradas: s → 5000 → n.

Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
0 (inicial)	continuar = "s" saldo = 250000	Todavía no se evalúa.	—	saldo = 250000; contador = 0; total = 0. Se lee continuar = "s".	contador = 0 total = 0	Bienvenida y saldo inicial.
1	continuar = "s" saldo = 250000	"s" == "s" Y 250000 >= 5000: verdadero.	Sí	monto = 5000. Fuera de límites: falso. 5000 > 250000: falso. saldo: 250000 - 5000 = 245000 contador: 0 + 1 = 1 total: 0 + 5000 = 5000 245000 >= 5000: verdadero. continuar: "s" → "n".	contador = 1 total = 5000	Retiro realizado. Saldo: ₡245 000. Pregunta si desea continuar.
Final	continuar = "n" saldo = 245000	"n" == "s": falso.	No	Valores sin cambios.	contador = 1 total = 5000	Saldo: ₡245 000. Retiros: 1. Total: ₡5 000.

Causa de terminación: el usuario responde "n" después del primer retiro.

Traza del caso límite

Entradas: s → 100000 → s → 100000 → s → 50000. Se prueba el máximo por operación y se agota el saldo.

Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
0 (inicial)	continuar = "s" saldo = 250000	Todavía no se evalúa.	—	saldo = 250000; contador = 0; total = 0. Se lee continuar = "s".	contador = 0 total = 0	Bienvenida y saldo inicial.
1	continuar = "s" saldo = 250000	"s" == "s" Y 250000 >= 5000: verdadero.	Sí	monto = 100000. Fuera de límites: falso. 100000 > 250000: falso. saldo: 250000 - 100000 = 150000 contador: 0 + 1 = 1 total: 0 + 100000 = 100000 150000 >= 5000: verdadero. continuar: "s" → "s".	contador = 1 total = 100000	Retiro realizado. Saldo: ₡150,000. Pregunta si desea continuar.
2	continuar = "s" saldo = 150000	"s" == "s" Y 150000 >= 5000: verdadero.	Sí	monto = 100000. Fuera de límites: falso. 100000 > 150000: falso. saldo: 150000 - 100000 = 50000 contador: 1 + 1 = 2 total: 100000 + 100000 = 200000 50000 >= 5000: verdadero. continuar: "s" → "s".	contador = 2 total = 200000	Retiro realizado. Saldo: ₡50,000. Pregunta si desea continuar.
3	continuar = "s" saldo = 50000	"s" == "s" Y 50000 >= 5000: verdadero.	Sí	monto = 50000. Fuera de límites: falso. 50000 > 50000: falso. saldo: 50000 - 50000 = 0 contador: 2 + 1 = 3 total: 200000 + 50000 = 250000 0 >= 5000: falso. No se pide respuesta; continuar conserva "s".	contador = 3 total = 250000	Retiro realizado. Saldo: ₡0. El saldo no alcanza para otro retiro.
Final	continuar = "s" saldo = 0	"s" == "s": verdadero. 0 >= 5000: falso. Resultado: falso.	No	No se ejecuta el cuerpo. Valores sin cambios.	contador = 3 total = 250000	Saldo: ₡0. Retiros: 3. Total: ₡250 000.

Causa de terminación: saldo = 0, que es menor que RETIRO_MINIMO. No hace falta cambiar continuar a "n".

Resultados de las pruebas del programa

Se ejecutó cajero_automatico.py con ocho conjuntos de datos. Cada ejecución comienza con un saldo de ₡250 000. Las flechas indican el orden de las respuestas ingresadas. Los resultados reales coincidieron con los resultados esperados.

Caso	Entradas	Resultado esperado	Resultado real	¿Coincide?
1	s → 20000 → s → 30000 → n	Saldo: ₡200,000; retiros: 2; total retirado: ₡50,000.	Dos retiros aceptados. Saldo: ₡200,000; retiros: 2; total: ₡50,000.	Sí
2	n	Saldo: ₡250,000; retiros: 0; total retirado: ₡0.	No se solicita monto. Saldo: ₡250,000; retiros: 0; total: ₡0.	Sí
3	s → 5000 → n	Saldo: ₡245,000; retiros: 1; total retirado: ₡5,000.	Retiro mínimo aceptado. Saldo: ₡245,000; retiros: 1; total: ₡5,000.	Sí
4	s → 100000 → s → 100000 → s → 50000	Saldo: ₡0; retiros: 3; total retirado: ₡250,000.	Se agota el saldo; termina automáticamente. Saldo: ₡0; retiros: 3; total: ₡250,000.	Sí
5	s → 4000 → n	Saldo: ₡250,000; retiros: 0; total retirado: ₡0.	Monto inferior al mínimo rechazado. Saldo: ₡250,000; retiros: 0; total: ₡0.	Sí
6	s → 100001 → n	Saldo: ₡250,000; retiros: 0; total retirado: ₡0.	Monto superior al máximo rechazado. Saldo: ₡250,000; retiros: 0; total: ₡0.	Sí
7	s → 100000 → s → 100000 → s → 60000 → n	Saldo: ₡50,000; retiros: 2; total retirado: ₡200,000.	Tercer retiro rechazado por fondos insuficientes. Saldo: ₡50,000; retiros: 2; total: ₡200,000.	Sí
8	s → 100000 → s → 100000 → s → 47000	Saldo: ₡3,000; retiros: 3; total retirado: ₡247,000.	Quedan ₡3 000; termina automáticamente. Saldo: ₡3,000; retiros: 3; total: ₡247,000.	Sí

Revisión del código

El programa usa nombres descriptivos, constantes en mayúsculas al inicio, indentación de cuatro espacios, mensajes claros, condicionales y un ciclo while. El contador y el acumulador sólo cambian con retiros válidos. Se espera que el monto ingresado sea numérico y que la respuesta sea s o n. El ciclo termina con n o cuando el saldo es menor al retiro mínimo.